PRD: espblynk

Overview

Create firmware for an ESP32-C3 mini development board that reads an MPU6050 and DHT22, then publishes sensor values to the project's existing Blynk IoT dashboard over Wi-Fi.

Goals

- Read temperature, humidity, acceleration, and orientation values from the connected sensors.
- Publish all eight values to their assigned Blynk datastreams every two seconds.
- Maintain a usable connection to Blynk Cloud and resume publishing after connectivity interruptions.

Target Hardware

- **Board:** ESP32-C3-DevKitM-1
- **MPU6050 I²C:** SCL on GPIO2; SDA on GPIO3
- **DHT22:** data on GPIO1 (board pin 10)
- **Blynk:** Template ID `TMPL3VjCwAzY1`; template name `ESPInterface`
- Wi-Fi and Blynk credentials were supplied and must be provided to the firmware as configuration secrets. Do not commit passwords or authentication tokens to a public source repository.

Functional Requirements

- Read DHT22 temperature and humidity.
- Read MPU6050 acceleration and report each axis in m/s².
- Calculate gyroscope-derived orientation angles in degrees, using accelerometer-assisted correction to reduce drift.
- Publish all readings every two seconds using these virtual pins:

Value	Blynk pin
Temperature	V0
Humidity	V1
Accelerometer X	V2
Accelerometer Y	V3
Accelerometer Z	V4
Gyroscope orientation X	V5
Gyroscope orientation Y	V6
Gyroscope orientation Z	V7

- Connect to Blynk IoT over Wi-Fi using the supplied configuration.
- Handle sensor read failures and network disconnections without terminating the measurement loop; reconnect and resume publishing when possible.

Non-Functional Requirements

- Keep the two-second sampling and publishing cadence during normal operation.
- Keep credentials out of source files intended for sharing or version control.

- Use accelerometer correction for tilt axes where applicable. Accelerometer correction cannot establish an absolute yaw reference, so the Z orientation may drift over time.

Constraints

- Use the specified GPIO assignments and Blynk virtual pin mapping.
- The dashboard and datastreams already exist; firmware is responsible for sending values, not creating the dashboard.
- The MPU6050's I²C address, sensor calibration procedure, and orientation-filter parameters have not been specified.

Open Questions

- Which MPU6050 I²C address should be used if it is not reliably detected automatically?
- Should orientation angles be zeroed at startup, and what startup pose should define zero?
- Which build framework and method for supplying local Wi-Fi/Blynk secrets should be used?